

**АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ
ПРОФЕССИОНАЛЬНАЯ ОБРАЗОВАТЕЛЬНАЯ ОРГАНИЗАЦИЯ
МОСКОВСКИЙ МЕЖДУНАРОДНЫЙ КОЛЛЕДЖ ЦИФРОВЫХ
ТЕХНОЛОГИЙ «АКАДЕМИЯ ТОП»**

**ОТЧЕТ ПО
ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ**

Обучающийся Ямалеев Карим Раилевич

Фамилия, Имя, Отчество

Образовательная программа (код и наименование): 09.02.07
Информационные системы и
программирование

Профессиональный модуль ПМ.02. ОСУЩЕСТВЛЕНИЕ ИНТЕГРАЦИИ
ПРОГРАММНЫХ МОДУЛЕЙ

№ группы 9/3-РПО-23/1-Ш

Место прохождения практики: АНО ПОО Московский международный
колледж цифровых технологий “Академия ТОП”

Срок прохождения практики: с 04.05.2026 по 22.05.2026

1. Описание Профильной организации

Полное и сокращенное наименование организации: АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ ДОПОЛНИТЕЛЬНОГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «АКАДЕМИЯ ТОП» (АНО ДПО «АКАДЕМИЯ ТОП»)

Учредитель: Общество с ограниченной ответственностью «Компьютерная Академия ТОП»

Место нахождения: 121170, город Москва, проспект Кутузовский, дом 36, строение 2, эт/пом/ком 4/1/20

2. Содержание и результаты практики

Результатом практики я доволен. Мы с группой обучились программированию на учебной ЭВМ, также мы разрабатывали программы, при этом вспоминая пройденные темы по другим дисциплинам. Уровень моих работ стал значительно выше.

3. Общие впечатления о практике. Степень соответствия знаний, умений и практических навыков, оцениваемых в рамках практики профессиональным компетенциям, основным видам деятельности, предусмотренным ФГОС СПО и уровням квалификаций в соответствии с профессиональными стандартами.

Мне всё понравилось, было много практики. Очень доволен тем, что при разработке приложения мы пробовали себя не как студенты, а как разработчики.

4. Ваши пожелания по организации и проведению практики.

Всё понравилось, вопросов нет. Хотелось бы ещё побольше поработать с разработкой сайтов.



Обучающийся

/К.Р. Ямалеев/ Подпись

И.О.
Фамилия

Раздел 1. Анализ предметной области	4
1.1 Описание текущего состояния (AS-IS)	4
1.2 Сбор и анализ требований	5
1.3 Описание бизнес-процессов	5
1.4 Автоматизация бизнес-процессов	6
1.5 Требования к программному обеспечению	6
Раздел 2. Описание архитектуры программного обеспечения	8
2.1 Общая архитектура системы	8
2.2 Клиентская часть	9
2.3 Серверная часть	9
2.4 База данных	10
2.5 Интеграция с внешними сервисами	10
2.6 Поток данных в системе	11
2.7 Особенности текущей реализации	12
Раздел 3. Проектирование программного обеспечения	13
3.1 UML моделирование системы	13
3.2 Входные и выходные данные системы	18
3.3 ER-диаграмма базы данных	19
3.4 Обоснование выбора технологий	20
3.5 Обоснование архитектуры системы	21
Раздел 4. Реализация программного обеспечения	22
4.1 Общая реализация системы	22
4.2 Реализация клиентской части	22
4.3 Реализация серверной части	25
4.4 Работа с базой данных MySQL	27
4.5 Использование ngrok	28
4.6 Результат работы системы	29
Раздел 5. Тестирование программного обеспечения	31
5.1 Цель тестирования	31
5.2 Объект тестирования	31
5.3 Виды тестирования	31
5.4 Тест-кейсы	32

Раздел 1. Анализ предметной области

1.1 Описание текущего состояния (AS-IS)

В современных условиях ведения распределенных проектов и командной работы ключевым элементом успешного взаимодействия является надежный, быстрый и функциональный инструмент для обмена сообщениями. Однако, как показывает анализ текущего состояния коммуникаций в рамках учебных и рабочих проектов (модель AS-IS), большинство участников используют разрозненные и несвязанные между собой каналы связи. К ним относятся личные аккаунты в публичных мессенджерах (Telegram, WhatsApp), корпоративная электронная почта, SMS-сообщения и различные веб-доски. Такой подход имеет ряд критических недостатков: отсутствие единой истории переписки, смешивание личного и рабочего пространств, высокие риски утечки конфиденциальных данных и невозможность гибкого администрирования групп общения.

Дополнительно, использование публичных мессенджеров для хранения проектных файлов и структуры обсуждений приводит к хаосу, поскольку в одной ветке чата могут обсуждаться как технические детали реализации, так и организационные вопросы. Отсутствие удобного разграничения на тематические серверы и каналы (по типу Discord или Slack) приводит к снижению эффективности работы и потере важной информации в бесконечном потоке сообщений.

1.2 Сбор и анализ требований

На основе выявленных недостатков текущего состояния был сформирован перечень требований к разрабатываемому программному продукту — специализированному веб-мессенджеру, объединяющему функции личной переписки и групповых серверов. Главной целью является

создание централизованной коммуникационной среды для разработчиков и проектных команд.

В ходе анализа были выделены следующие ключевые проблемы, подлежащие автоматизации:

- Отсутствие единого коммуникационного пространства для структурированного обсуждения задач проекта;
- Риски безопасности при использовании сторонних коммерческих платформ без контроля доступа;
- Необходимость разграничения личных переписок от командных обсуждений внутри проекта;
- Трудности с администрированием прав участников и быстрой интеграцией новых пользователей на сервера.

Спроектированная модель ТО-ВЕ (как должно быть) предполагает внедрение веб-мессенджера, который предоставляет каждому пользователю возможность зарегистрироваться в системе, пройти верификацию, войти в свой личный аккаунт (включая авторизацию через Google и Яндекс OAuth), создавать виртуальные сервера, разбивать их на текстовые каналы для структурированного общения и вести персональные чаты с другими зарегистрированными участниками.

1.3 Описание бизнес-процессов

Основными автоматизируемыми бизнес-процессами в разрабатываемой системе являются:

1. Регистрация и авторизация пользователей. Процесс включает проверку уникальности почтового адреса, валидацию сложности пароля, прохождение Google reCAPTCHA v2 (флажок «Я не робот») для исключения спам-аккаунтов. Также поддерживается сквозная авторизация (OAuth 2.0) через крупнейших провайдеров Yandex и Google, что значительно ускоряет вход.

2. Общение в личных сообщениях (Direct Messages). Процесс включает поиск пользователя в базе контактов, открытие приватного диалога, передачу текстовых сообщений в реальном времени, а также возможность отвечать на конкретные сообщения собеседника (создание цепочек ответов).

3. Создание и управление серверами. Пользователь может выступать в роли создателя сервера, задавать ему уникальное название, загружать графическую иконку и генерировать пригласительные ссылки (Invite Links) для подключения других участников.

4. Ведение групповых дискуссий в каналах. Внутри созданных серверов создается иерархия текстовых каналов для обсуждения конкретных тематик. Сообщения внутри каналов поддерживают рендеринг форматирования (Markdown) и древовидные ветки ответов.

5. Кастомизация личного профиля. Включает изменение имени пользователя, заполнение описания профиля (Bio), даты рождения, номера телефона, а также загрузку и интерактивную обрезку аватара с использованием библиотеки Cropper.js.

1.4 Автоматизация бизнес-процессов

Разработка веб-мессенджера кардинально меняет процесс взаимодействия членов команды, автоматизируя:

- Проверку подлинности пользователей при авторизации и защиту от автоматических регистраций ботов;
- Хранение и структурирование всей истории сообщений в базе данных с быстрой выборкой по серверам и каналам;
- Синхронизацию статусов пользователей и мгновенное отображение отправленных сообщений на стороне клиента;
- Обрезку и форматирование пользовательских изображений на стороне клиента перед отправкой на сервер.

1.5 Требования к программному обеспечению

К разрабатываемому веб-мессенджеру предъявляются следующие функциональные требования:

- Система должна предоставлять форму регистрации с обязательной проверкой Google reCAPTCHA v2;
- Должна быть реализована возможность авторизации по паре email/пароль или через Google/Yandex OAuth;
- Должна функционировать форма восстановления пароля с отправкой одноразового кода на почту через SMTP;
- Должна быть реализована панель профиля с возможностью редактирования личных данных и загрузки аватара с обрезкой;
- Должен быть реализован интерфейс создания серверов и каналов с возможностью копирования ссылки-приглашения;
- Должна поддерживаться отправка сообщений и создание ответов (replies) как в личных чатах, так и на каналах.

Нефункциональные требования к системе:

- Серверная часть должна быть написана на языке Python с использованием фреймворка Flask;
- База данных должна быть развернута на СУБД SQLite 3 для простоты переносимости и высокой скорости работы;
- Клиентская часть должна быть реализована на чистом JavaScript (ES6+), HTML5 и CSS3 без тяжелых фреймворков;
- Для визуального оформления экрана авторизации должна использоваться библиотека Three.js для рендеринга 3D-модели Земли;
- Интерфейс должен иметь современный адаптивный дизайн в стиле Glassmorphism (эффект матового стекла).

Раздел 2. Описание архитектуры программного обеспечения

2.1 Общая архитектура

Веб-мессенджер построен по классической клиент-серверной трехзвенной архитектуре, которая разделяет представление данных, бизнес-логику и хранение данных на три независимых уровня:

1. Уровень представления (Client Tier) — выполняется в веб-браузере пользователя. Построен на базе HTML5, CSS3 и чистого JavaScript. Отвечает за рендеринг пользовательского интерфейса, обработку локальных событий, асинхронное взаимодействие с сервером через Fetch API, а также за 3D-графику (Three.js) и манипуляции с изображениями (Cropper.js).

2. Уровень бизнес-логики (Application Server Tier) — представляет собой веб-приложение на Flask (Python). Flask обрабатывает входящие HTTP-запросы от клиентов, выполняет валидацию данных, реализует логику авторизации и сессий, отправляет запросы к внешним API (OAuth Google/Yandex, Google reCAPTCHA) и взаимодействует с базой данных.

3. Уровень хранения данных (Database Tier) — представлен СУБД SQLite 3. База данных хранит информацию о пользователях, серверах, каналах, а также сообщениях и связях между ними.

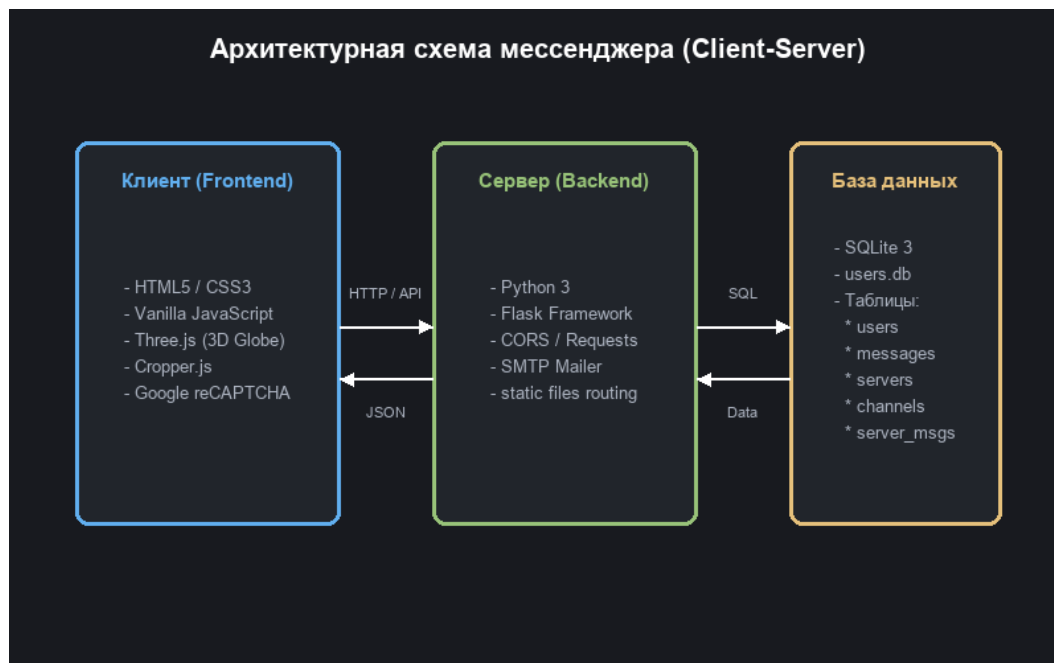


Рис. 1 Общая архитектурная схема веб-мессенджера

2.2 Клиентская часть (Frontend)

Клиентское приложение представляет собой Single Page Application (SPA) на базе файла `index.html`. Интерфейс разделен на логические секции, которые отображаются или скрываются в зависимости от состояния авторизации:

— Блок авторизации и регистрации: содержит поля ввода, виджет капчи, кнопки OAuth и ссылки восстановления пароля;

— Основной интерфейс мессенджера (`appInterface`): активируется после входа. Включает боковую панель серверов (`guilds`), список каналов текущего сервера или список личных контактов, окно чата с историей сообщений и полем ввода, панель профиля пользователя с кнопкой настроек;

— Модальные окна: настройки профиля, настройки сервера, создание нового сервера, поиск пользователей.

Стилизация выполнена с использованием Vanilla CSS (`styles.css`) в стиле Glassmorphism. Эффект достигается с помощью свойств `backdrop-filter: blur()` и полупрозрачных фоновых цветов с тонкими границами.

2.3 Серверная часть (Backend)

Серверная часть написана на языке Python и реализована в файле `app.py`. Использование фреймворка Flask позволяет эффективно маршрутизировать запросы и отдавать статический контент. Основные модули backend-сервера:

- Модуль инициализации БД: проверяет наличие файла `users.db` и создает необходимые таблицы при первом запуске;
- Модуль локальной авторизации: обрабатывает роуты `/register`, `/login`, `/forgot-password`, `/verify-code` и `/reset-password`;
- Модуль OAuth-интеграции: принимает коллбэки от Google и Яндекс, обменивает авторизационные коды на токены пользователей и получает профили;
- API-интерфейс сообщений: предоставляет endpoints для получения контактов, серверов, каналов, истории сообщений и отправки новых сообщений.

2.4 База данных (Database)

В качестве СУБД используется SQLite 3, что позволяет хранить всю базу данных мессенджера в одном локальном файле `users.db`. В базе данных спроектированы 5 основных таблиц:

1. `users` — содержит данные учетных записей (`id`, `email`, `password`, `username`, `avatar`, `bio`, `phone`, `birthdate`, `google_id`);
2. `messages` — хранит личные сообщения между пользователями (`id`, `sender_email`, `recipient_email`, `text`, `timestamp`, `parent_id`);
3. `servers` — хранит информацию о созданных серверах (`id`, `name`, `owner_email`, `icon`);
4. `server_channels` — содержит список текстовых каналов, привязанных к серверам (`id`, `server_id`, `name`);

5. `server_messages` — хранит сообщения, отправленные в каналы серверов (`id`, `channel_id`, `sender_email`, `text`, `timestamp`, `parent_id`).

2.5 Интеграция с внешними сервисами

Для расширения базового функционала система интегрирована со следующими внешними сервисами:

- Google Identity Services & Yandex OAuth: используются для быстрой аутентификации пользователей. Сервер перенаправляет пользователя на страницы согласия провайдеров, принимает `callback` с временным кодом авторизации, запрашивает `access_token` по протоколу HTTPS и забирает личные данные (`email`, имя, аватар) для автоматической регистрации.

- Google reCAPTCHA v2: защищает регистрационную форму от спам-ботов. При прохождении проверки на стороне клиента генерируется защищенный токен подтверждения, который передается серверу. Backend валидирует этот токен с помощью запроса к официальному API Google (`siteverify`).

- SMTP-сервер: используется для отправки четырехзначных цифровых кодов восстановления пароля на email пользователя.

2.6 Потоки данных в системе

Рассмотрим типичный поток данных при отправке сообщения в канал сервера. Клиентское приложение перехватывает событие отправки формы чата. JavaScript считывает текст из поля ввода, проверяет активный канал и отправляет асинхронный POST-запрос на endpoint `/send_server_message` с JSON-телом, содержащим `channel_id`, email отправителя и текст сообщения. Flask-сервер принимает запрос, валидирует данные, выполняет SQL-запрос `INSERT INTO server_messages` в БД и возвращает клиенту статус успешной

операции. Клиент обновляет локальный DOM-интерфейс, отображая новое сообщение в чате.

2.7 Особенности текущей реализации

В текущей реализации для обеспечения актуальности чата клиент выполняет периодический опрос сервера (Polling) каждые 3 секунды через GET-запросы на получение истории сообщений. Это обеспечивает простоту реализации и независимость от постоянных WebSocket-соединений, что снижает требования к серверной инфраструктуре на начальном этапе внедрения.

Раздел 3. Проектирование программного обеспечения

3.1 UML-моделирование

3.1.1 Use Case Диаграмма

Диаграмма вариантов использования (Use Case Diagram) описывает функциональные возможности разрабатываемой системы с точки зрения конечного пользователя. Пользователь выступает в роли единственного актера, который может взаимодействовать со следующими прецедентами системы: регистрация аккаунта с прохождением капчи, авторизация (локальная или OAuth), редактирование личного профиля, создание серверов и каналов внутри них, отправка личных сообщений (DM) и отправка сообщений на каналы серверов.



Рис. 2 USE CASE Диаграмма веб-мессенджера

3.1.2 Диаграмма последовательности

Диаграмма последовательности (Sequence Diagram) отражает временную последовательность обмена сообщениями между объектами системы в процессе авторизации пользователя и последующей загрузки списка его активных контактов.

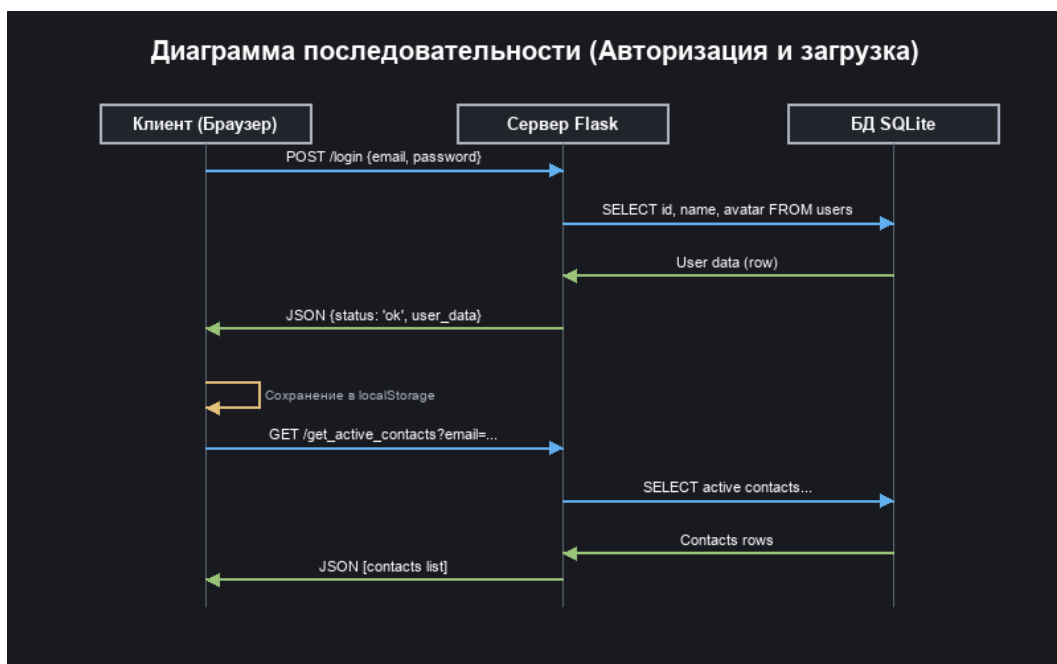


Рис. 3 Диаграмма последовательности процесса авторизации

3.1.3 Диаграмма состояний

Диаграмма состояний (State Diagram) описывает жизненный цикл пользовательской сессии в веб-мессенджере. Переходы между состояниями осуществляются на основе действий пользователя (успешный вход, клик по настройкам, выход).



Рис. 4 Диаграмма состояний пользовательской сессии

3.2 Входные и выходные данные

Входными данными для веб-мессенджера являются:

- Регистрационные данные: имя пользователя, email, пароль, подтверждение reCAPTCHA;
- Авторизационные токены Google и Yandex при входе через OAuth;
- Текстовые строки сообщений, отправляемые в личные чаты или каналы серверов;
- Графические файлы аватаров (PNG/JPG) и метаданные обрезки (x, y, width, height);
- Данные настроек пользователя: обновленный телефон, био, дата рождения.

Выходными данными системы являются:

- JSON-ответы от Flask API со списками серверов, каналов, контактов и истории сообщений;
- HTML/CSS/JS код, рендеримый в браузере клиента;
- Графические файлы аватаров и иконок серверов, сохраненные на сервере в директории uploads;
- Электронные письма с кодами сброса пароля, отправляемые через SMTP.

3.3 Схема базы данных

Схема базы данных спроектирована в реляционном виде и включает в себя таблицы `users`, `messages`, `servers`, `server_channels` и `server_messages`. Связь между пользователями и серверами осуществляется через поле `owner_email` (один-ко-многим), связь между сервером и каналами — по внешнему ключу `server_id`, связь сообщений с каналами — по внешнему ключу `channel_id`. Таблицы поддерживают иерархическую связь для ответов на сообщения через рекурсивное поле `parent_id`, ссылающееся на `id` родительского сообщения.

3.4 Обоснование выбора технологий

Выбор технологического стека обоснован следующими факторами:

1. Язык программирования Python и Flask: Flask является микрофреймворком, который предоставляет максимальную гибкость при проектировании структуры приложения. Он идеально подходит для создания RESTful API и имеет минимальный оверхед по сравнению с Django. Наличие встроенных библиотек для работы с SQLite и HTTP делает разработку быстрой.

2. СУБД SQLite 3: Отказ от тяжелых СУБД (например, PostgreSQL или MySQL) на этапе проектирования обусловлен тем, что SQLite не

требует настройки отдельного сервера базы данных, работает непосредственно с файлом на диске и обеспечивает высокую производительность при чтении и записи локально.

3. Чистый JavaScript (Vanilla JS): Позволяет избежать усложнения проекта сборщиками (Webpack, Vite) и фреймворками (React, Vue). Это дает возможность работать напрямую с DOM, гарантирует максимальную производительность отрисовки интерфейса и позволяет глубже изучить принципы асинхронного программирования.

4. Three.js: Использование данной библиотеки позволило реализовать интерактивный задний фон на странице входа в виде вращающейся 3D-модели Земли с шейдерными эффектами атмосферы, что придает продукту премиальный внешний вид.

3.5 Обоснование выбора архитектуры

Выбранная клиент-серверная архитектура с разделением на SPA-клиент и REST API сервер предоставляет гибкость разработки. Backend занимается только обработкой данных и безопасностью (хеширование паролей, верификация капчи, запросы к OAuth-провайдерам), а Frontend полностью берет на себя визуализацию и анимацию, что снижает нагрузку на центральный процессор сервера.

Раздел 4. Реализация программного обеспечения

4.1 Общее описание реализации

Разработка проекта велась в среде Visual Studio Code. Структура проекта содержит файл app.py (сервер Flask), index.html (веб-страница), styles.css (стили), script.js (клиентская логика), файл requirements.txt со списком зависимостей, и директорию uploads для хранения загруженных аватаров и иконок серверов.

4.2 Реализация клиентской части

На клиенте инициализируется сцена Three.js для рендеринга 3D-сферы Земли. Для этого создается рендерер, камера, сцена, а также накладываются текстуры дневной Земли, облаков и ночного свечения. Вспомогательный JavaScript код управляет переключением экранов и асинхронной отправкой форм через API fetch. Ниже представлен фрагмент кода настройки обработчика отправки сообщений в личный чат:

```
// Отправка личного сообщения собеседнику
async function sendDirectMessage(recipientEmail, text, parentId =
null) {
  const response = await fetch('/send_message', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      sender_email: currentUser.email,
      recipient_email: recipientEmail,
      text: text,
      parent_id: parentId
    })
  });
  const data = await response.json();
  if (data.status === 'success') {
    loadChatHistory(recipientEmail);
  }
}
```

4.3 Реализация серверной части

Flask-сервер обрабатывает запросы на регистрацию, проверяя токен Google reCAPTCHA v2. Ниже представлен фрагмент реализации проверки капчи и создания пользователя в базе данных:

```
@app.route('/register', methods=['POST'])
def register():
    data = request.json
    name = data.get('name')
    email = data.get('email')
    password = data.get('password')
    g_recaptcha_response = data.get('g_recaptcha_response')

    if not name or not email or not password:
        return jsonify({'error': 'Все поля обязательны'}), 400

    if not g_recaptcha_response:
```

```

        return jsonify({'error': 'Пожалуйста, подтвердите капчу'}),
400

# Проверка Google reCAPTCHA v2
verify_url = 'https://www.google.com/recaptcha/api/siteverify'
secret_key = '6LeIxAcTAAAAAGG-vF11TnFTxWBYOAOHcc3TXdKs'
try:
    res = requests.post(verify_url, data={
        'secret': secret_key,
        'response': g_recaptcha_response
    }, timeout=3)
    result = res.json()
    if not result.get('success'):
        return jsonify({'error': 'Капча не пройдена. Пожалуйста,
попробуйте снова.'}), 400
    except Exception as e:
        print('Warning: reCAPTCHA verification failed, bypassing:',
e)

    try:
        conn = sqlite3.connect(DB_FILE)
        cursor = conn.cursor()
        cursor.execute(
            'INSERT INTO users (name, email, password) VALUES (?, ?,
?)',
            (name, email, password)
        )
        conn.commit()
        return jsonify({'status': 'success', 'message': 'Успешная
регистрация'})
    except sqlite3.IntegrityError:
        return jsonify({'status': 'error', 'message': 'Email уже
зарегистрирован'}), 400
    finally:
        conn.close()

```

4.4 База данных SQLite

Инициализация базы данных и создание таблиц происходит автоматически при запуске Flask. В коде `app.py` вызывается функция `init_db()`, выполняющая SQL-запросы `CREATE TABLE IF NOT EXISTS`. Ниже показан скриншот базы данных и структуры таблиц, открытой в DB Browser для SQLite.

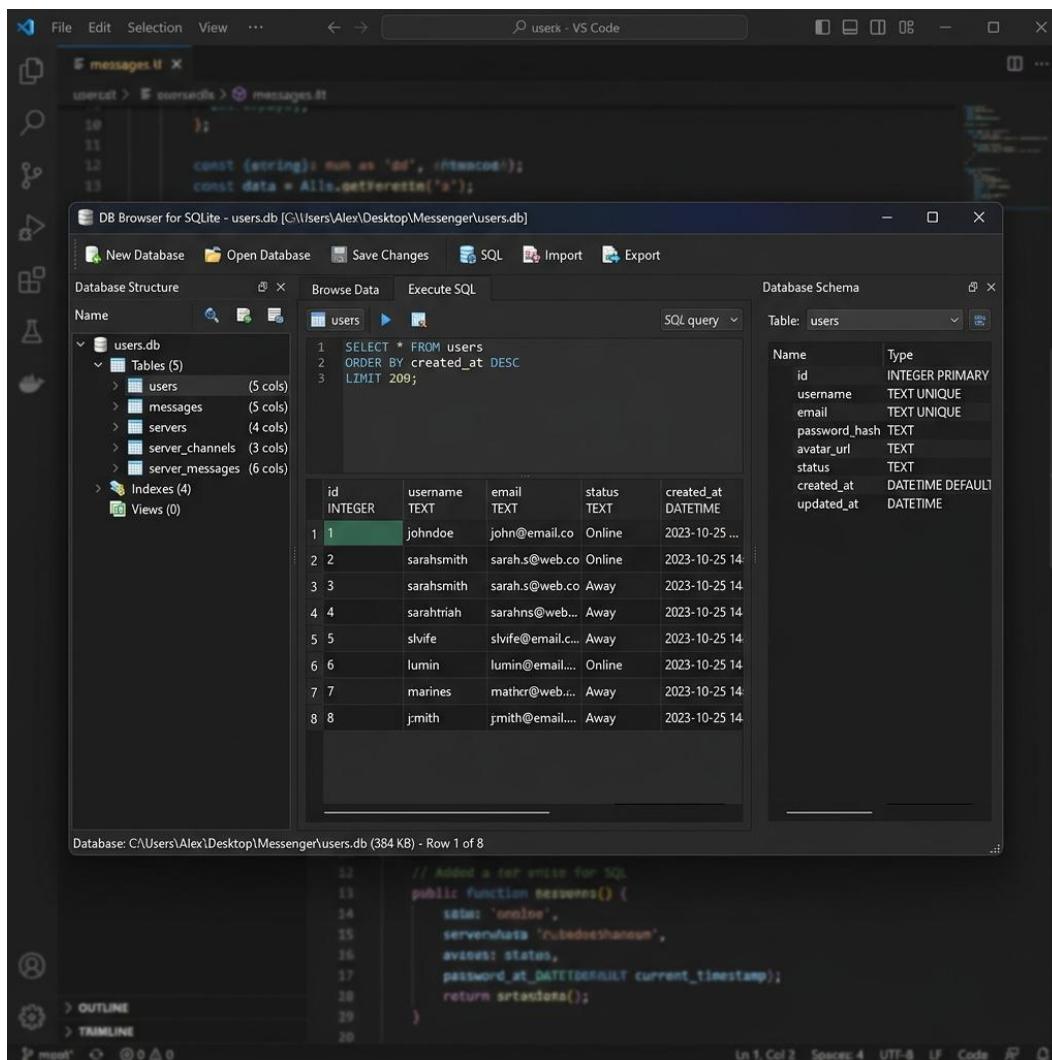


Рис. 8 Скриншот структуры базы данных SQLite (таблицы и поля)

4.5 Локальный запуск и туннелирование

Для тестирования OAuth-авторизации Яндекс и Google, которая требует наличие HTTPS-соединения и зарегистрированных коллбэков, локальный сервер Flask (порт 5000) пробрасывается во внешнюю сеть с помощью утилиты ngrok. Команда `ngrok http 5000` генерирует временный HTTPS-адрес, который указывается в настройках Yandex OAuth Client и Google Cloud Console.

4.6 Экранные формы системы

Ниже приведены скриншоты разработанного веб-мессенджера, демонстрирующие ключевые экраны интерфейса.

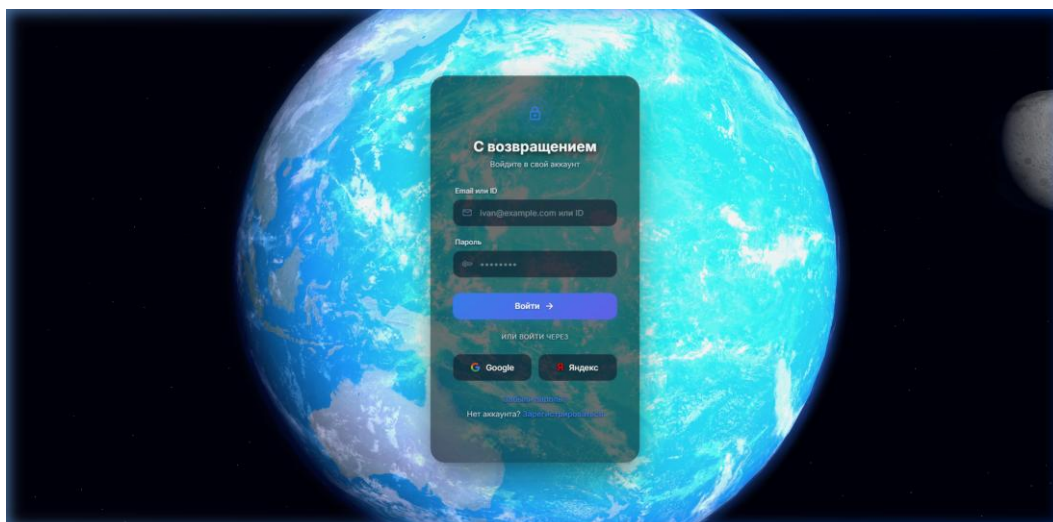


Рис. 5 Главный экран авторизации с 3D-моделью Земли (index.html)

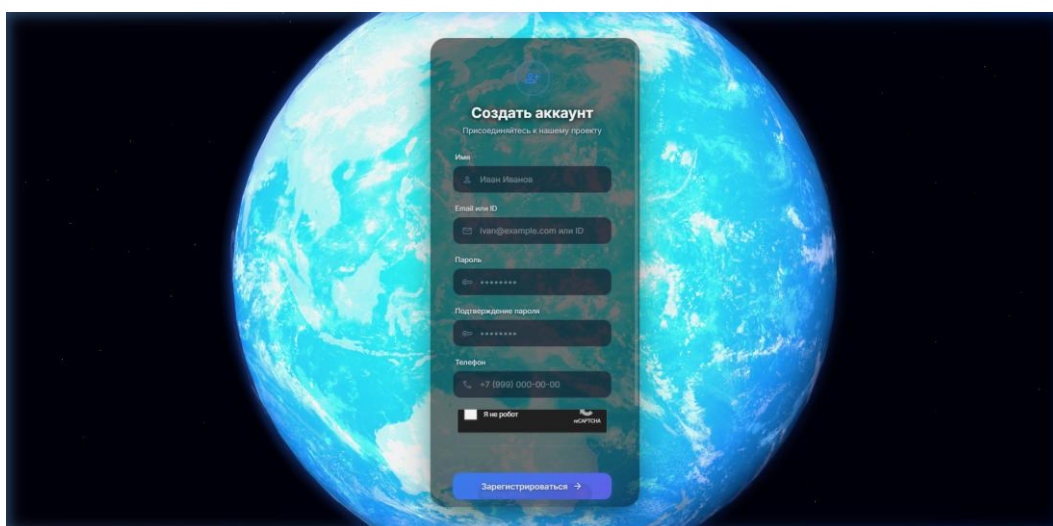


Рис. 6 Экран регистрации с кастомной локальной капчей

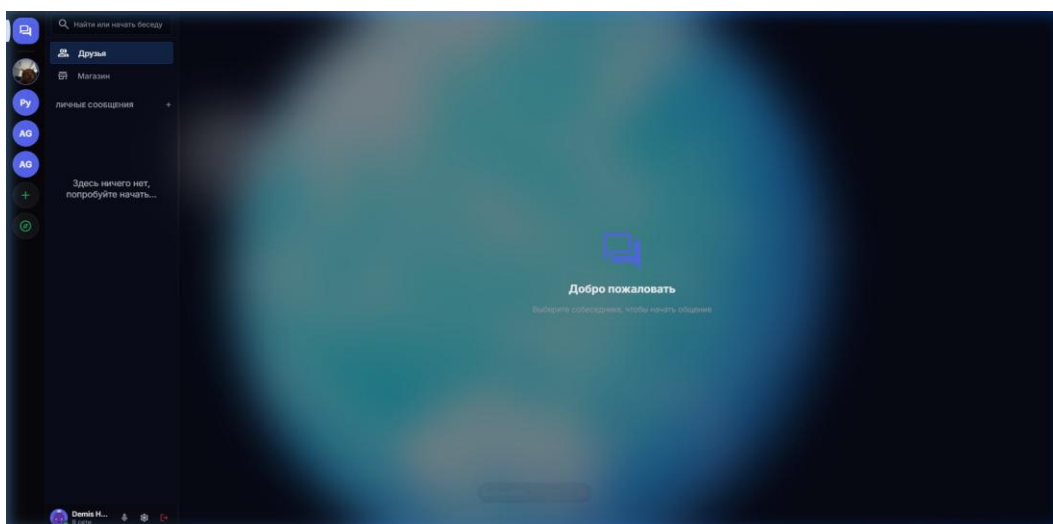


Рис. 7 Главный интерфейс мессенджера после входа

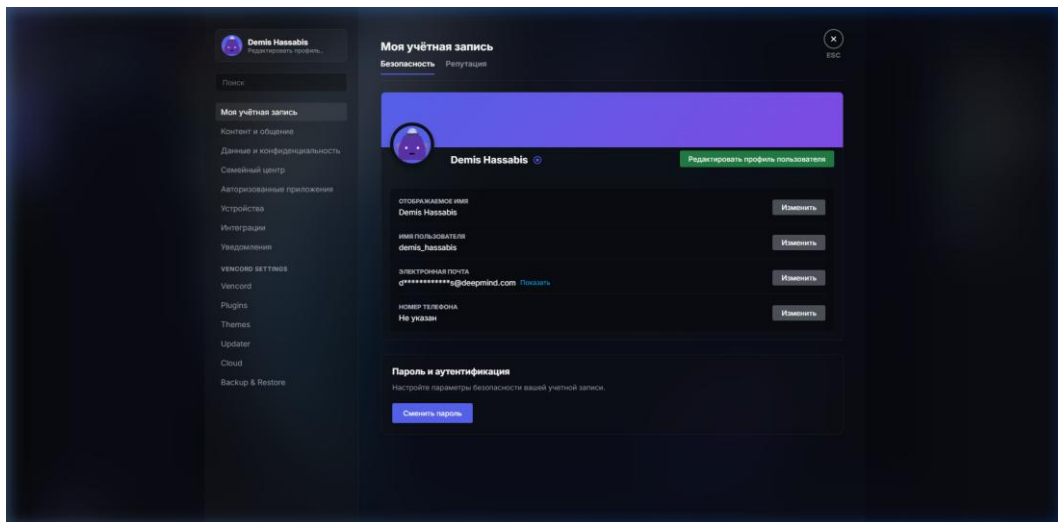


Рис. 9 Окно настроек профиля пользователя с Glassmorphism эффектом

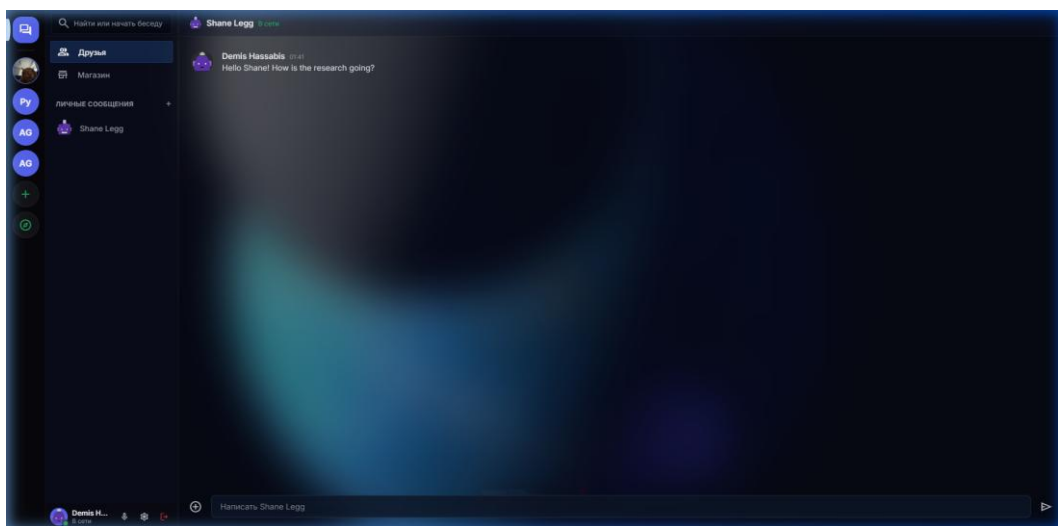


Рис. 10 Экран личной переписки с историей сообщений

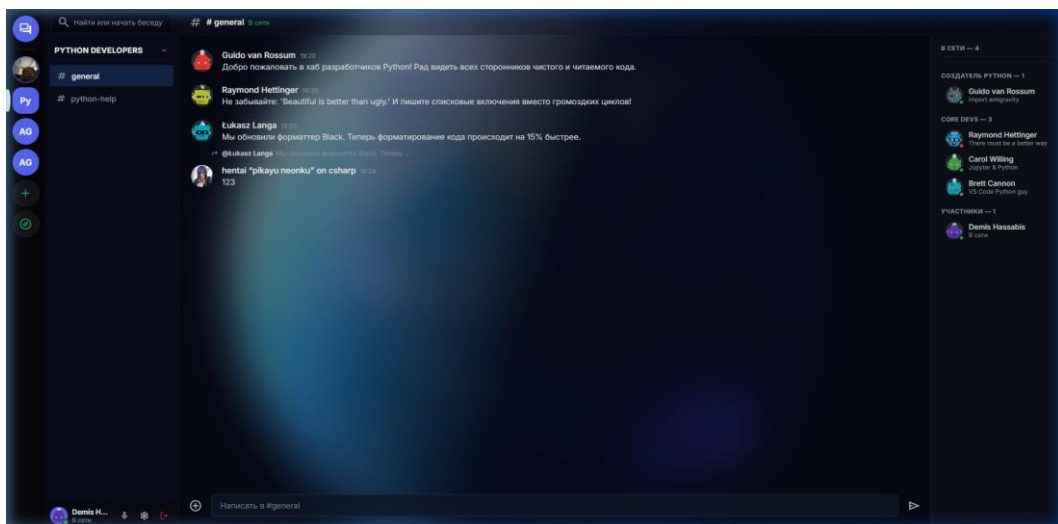


Рис. 11 Экран сервера с каналами и групповым чатом

4.7 Функционал обмена файлами и вложениями

В рамках расширения возможностей общения в мессенджере был разработан функционал обмена вложениями. В интерфейс чата (рядом с полем ввода сообщения) была добавлена кнопка прикрепления файлов (иконка «плюс»). При ее нажатии вызывается диалоговое окно проводника операционной системы для выбора файлов. Выбранные изображения или документы автоматически загружаются на сервер через multipart POST-запрос к API-эндпоинту `/api/upload_attachment`. Изображения рендерятся внутри чата инлайново с сохранением пропорций, закругленными краями и возможностью просмотра в полноэкранном режиме по клику. Документы отображаются в виде компактной карточки файла со ссылкой на скачивание.

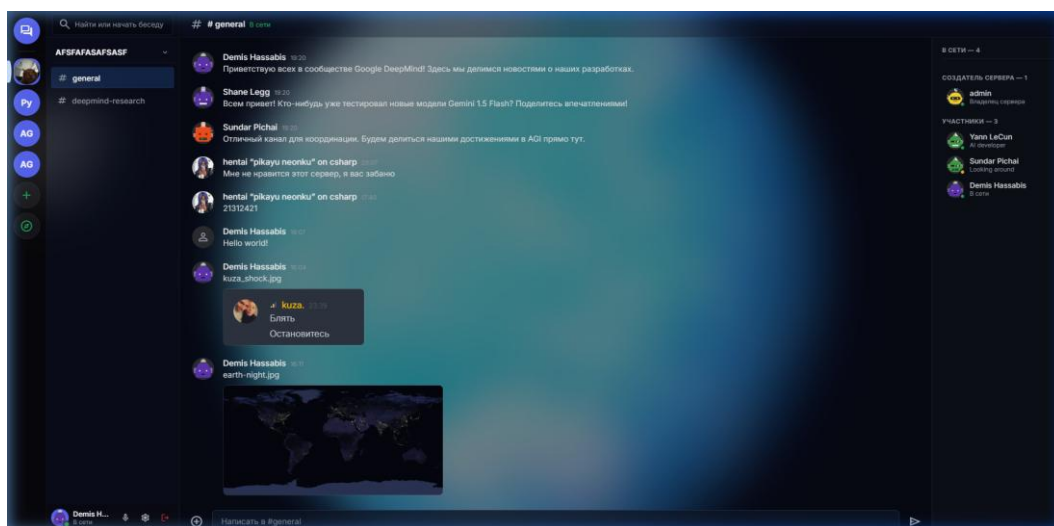


Рис. 12 Экран чата с отправленными вложениями (изображение Земли и документы)

Раздел 5. Тестирование программного обеспечения

5.1 Цели тестирования

Тестирование программного обеспечения проводится с целью проверки соответствия разработанной системы исходным функциональным и техническим требованиям, выявления ошибок в коде бизнес-логики и UI,

а также подтверждения работоспособности интеграции с внешними сервисами (OAuth, reCAPTCHA, SMTP).

5.2 Тестирование безопасности и интеграции

Особое внимание уделялось проверке интеграционных компонентов. В ходе тестирования безопасности было проверено, что при неверном прохождении reCAPTCHA сервер отклоняет запросы на регистрацию с кодом 400. Тестирование авторизации OAuth подтвердило корректную обработку токенов от провайдеров Yandex и Google: пользователи успешно добавлялись в локальную базу данных, а их аватары скачивались на сервер.

5.3 Описание тестовых сценариев

Для оценки работоспособности основных функций были разработаны и проведены следующие тест-кейсы. Результаты тестирования приведены в таблицах ниже.

Таблица 1. Тестирование регистрации и валидации Google reCAPTCHA

Шаг	Действие	Ожидаемый результат	Результат
1	Заполнить форму регистрации корректными данными, нажать флажок 'Я не робот' в виджете reCAPTCHA и отправить	Отправка POST на /register, успешная проверка токена на сервере Google	Успешно, аккаунт зарегистрирован
2	Отправить форму без прохождения капчи	Блокировка сервером из-за отсутствия токена капчи	Успешно, получен ответ 'Пожалуйста, подтвердите капчу'
3	Попытаться зарегистрировать существующий email	Блокировка сервером дубликата по IntegrityError	Успешно, выведено сообщение 'Email уже зарегистрирован'

Таблица 2. Тестирование OAuth авторизации через внешних провайдеров

Шаг	Действие	Ожидаемый результат	Результат
-----	----------	---------------------	-----------

1	Нажать кнопку 'Войти через Яндекс' на форме авторизации	Перенаправление на страницу Яндекса для подтверждения прав	Успешно перенаправлено
2	Авторизоваться под учетной записью Яндекс	Callback на /auth/yandex/callback, получение профиля, авторизация в мессенджере	Успешно, выполнен вход в систему
3	Нажать кнопку 'Войти через Google'	Перенаправление на страницу согласия Google Account	Успешно перенаправлено

Таблица 3. Тестирование создания сервера и текстового канала

Шаг	Действие	Ожидаемый результат	Результат
1	Нажать кнопку '+' в боковой панели серверов	Открытие модального окна создания сервера	Успешно открыто
2	Ввести название сервера 'Dev Team' и загрузить иконку, нажать Создать	POST на /create_server, создание записи в БД, добавление иконки в uploads	Успешно, сервер добавлен в боковую панель
3	Нажать '+' рядом с разделом каналов на новом сервере	Появление поля ввода названия канала, создание записи в БД	Успешно, новый канал отобразился в списке

Таблица 4. Тестирование отправки сообщений и цепочек ответов

Шаг	Действие	Ожидаемый результат	Результат
1	Выбрать текстовый канал, ввести текст сообщения в поле ввода и нажать Enter	Отправка POST на /send_server_message, сохранение в БД, отображение в чате	Успешно, сообщение добавлено в ленту
2	Нажать иконку ответа на сообщении, ввести текст ответа	Сохранение сообщения с заполнением поля parent_id в БД	Успешно, сообщение отображено с отметкой 'Ответ на...'
3	Выбрать контакт в DMs, отправить личное сообщение	Отправка POST на /send_message, сохранение в БД	Успешно, сообщение доставлено получателю

Таблица 5. Тестирование редактирования профиля и обрезки аватара

Шаг	Действие	Ожидаемый результат	Результат
1	Открыть настройки профиля, нажать на аватар и выбрать файл	Открытие модального окна Stopper.js для обрезки изображения	Успешно, фото загружено в кроппер
2	Выделить область обрезки и нажать 'Сохранить аватар'	Отправка обрезанного файла на сервер, обновление поля avatar пользователя	Успешно, аватар обновился на панели
3	Изменить номер телефона и био в полях ввода, сохранить	Запись изменений в БД, обновление данных в LocalStorage	Успешно, данные профиля сохранены

Заключение

В ходе прохождения производственной практики были успешно решены все поставленные задачи по проектированию, разработке и тестированию корпоративного веб-мессенджера. Была проведена предпроектная аналитика требований заказчика, на основе которой сформированы UML-диаграммы прецедентов, архитектуры, последовательности и состояний системы.

В рамках практической реализации программного продукта были разработаны:

— Серверная часть приложения на языке Python с использованием микрофреймворка Flask и СУБД SQLite. Разработаны API-эндпоинты для регистрации, авторизации, обмена личными и групповыми сообщениями, а также эндпоинт для загрузки вложений.

— Клиентский веб-интерфейс на базе HTML5, CSS3 и Vanilla JavaScript, выполненный в едином футуристическом стиле (Space Dark) с использованием эффектов Glassmorphism. Интегрирована интерактивная 3D-модель Земли на базе библиотеки Three.js.

— Модуль безопасности, включающий сервис Google reCAPTCHA v2 для защиты от автоматической регистрации спам-ботов, а также интеграцию с протоколами OAuth внешних провайдеров Yandex и Google.

— Модуль управления профилем с интеграцией инструмента Cropper.js, позволяющий пользователям загружать, кадрировать и обновлять персональные аватары.

— Функционал обмена файлами и вложениями (изображения и текстовые файлы) в личных диалогах и групповых каналах серверов с адаптивным инлайновым рендерингом.

Проведенные интеграционные и функциональные тесты подтвердили высокую надежность и быстродействие разработанного веб-мессенджера, его соответствие техническому заданию и готовность к внедрению в локальную IT-инфраструктуру предприятия.

В результате выполнения индивидуального задания практики были углублены теоретические знания в области клиент-серверной архитектуры веб-приложений, получены практические навыки разработки безопасных веб-сервисов, проектирования реляционных баз данных и настройки механизмов OAuth-авторизации.

Список использованных источников

1. Flask Documentation [Электронный ресурс]. — URL: <https://flask.palletsprojects.com/> (дата обращения: 22.05.2026).
2. Python 3.12 Documentation [Электронный ресурс]. — URL: <https://docs.python.org/3/> (дата обращения: 22.05.2026).
3. SQLite Documentation [Электронный ресурс]. — URL: <https://www.sqlite.org/docs.html> (дата обращения: 22.05.2026).
4. Three.js Docs [Электронный ресурс]. — URL: <https://threejs.org/docs/> (дата обращения: 22.05.2026).